Section 08: AI 連携

Gemini API を使って「知能」を組み込む

現代の Web 開発において、AI（人工知能）の活用は必須スキルです。今回は Google の **Gemini** を、Hono（バックエンド）経由で安全に呼び出す方法を学びます。

1. 準備：Google AI Studio

プログラミング不要で Gemini を試せるツールです。

1. **Google AI Studio** にアクセス。
2. 「Get API key」から、新しい API キーを発行します。

> **API キーは「家の鍵」と同じです。絶対に GitHub などに公開（コミット）してはいけません！**

2. セキュリティ：なぜバックエンドで叩くのか？

API キーを React（フロントエンド）に書いてしまうと、ブラウザの「検証」画面から誰でも盗めてしまいます。

- ❌ **フロントエンドから直接呼ぶ**: API キーが丸見え。悪用されるリスク大。
- ✅ **バックエンド (Hono) を経由する**: キーはサーバー側に隠し、フロントからは Hono を通して安全に AI に命令します。

これが **「バックエンド・ブリッジ」** という構成です。

3. 実装：Hono で API を作成する

まず、ライブラリをインストールします。

```
npm install @google/genai
```

次に、Hono で Gemini を呼び出すエンドポイントを作ります。

```
import { GoogleGenAI } from "@google/genai";

app.post("/ai/chat", async (c) => {
  // 環境変数から安全に API キーを取得 (Cloudflare Workers の Binding)
  const genAI = new GoogleGenAI({ apiKey: c.env.GEMINI_API_KEY });

  const { prompt } = await c.req.json();
  const response = await genAI.models.generateContent({
    model: "gemini-3.0-flash",
    contents: prompt,
  });

  return c.json({ response: response.text() });
});
```

4. 環境変数の設定 (wrangler secret)

API キーなどの機密情報は、ソースコードや設定ファイル（`wrangler.jsonc` など）に直接書いてはいけません。GitHub に push した瞬間に世界中に公開されてしまうからです。

Cloudflare Workers では、`wrangler` コマンドを使って安全にキーを保存します。

本番環境 (Wrangler Secret)

以下のコマンドを実行すると、暗号化された状態で Cloudflare に保存されます。

```
npx wrangler secret put GEMINI_API_KEY
# 実行後にキーの入力を求められるので、貼り付けて Enter
```

> `secret put` で保存した値は、ダッシュボードからも中身が見えないようになり、非常に安全です。

5. 応用アイデア

Gemini API を使えるようになると、アプリの可能性が無限に広がります。

- **要約機能**: 長いニュース記事を3行でまとめる。
- **感情分析**: ユーザーの口コミが「ポジティブ」か「ネガティブ」か判定する。
- **フォーマット変換**: ぐちゃぐちゃなメモを綺麗な JSON データに変換する。
- **画像解析**: 画像に何が写っているかを説明してもらう。

6. まとめ

AI 連携の本質は、**Part 7 で学んだ「フロント → バックエンド → 外部サービス」の通信パターンと同じ**です。
違いは、通信先がデータベースではなく Gemini API になっただけ。

1. フロントからバックエンドへリクエストを送る
2. バックエンドが API キーを使って Gemini を呼び出す
3. 結果をフロントに返す

この構成さえ理解していれば、Gemini 以外のどんな外部 API（Stripe, OpenAI, etc.）にも応用できます！